

# Git for Dummies — Clever Tricks & Recovery (Ivy\_Net)

---

Central doc index: `docs/README.md` — when to use this folder vs `sports|tenure|talent/documents/`.

Start here for concepts and first-time setup:

`GIT_MULTIPLE_MACHINES_ELEMENTARY.md` — three boxes (laptop / HPC / GitHub), `push` / `pull`, cloning Rivanna.

This file: copy-paste **recipes** for day-to-day work: two machines, dirty trees, bad staging, and “what actually happened?”

Replace `main` with your real default branch if you use something else.

---

## 1. Two-second health check

---

From the **repo root** (`Ivy_Net` — the folder that contains `sports/`, `tenure/`, `scripts/`):

```
git status -sb
git branch -vv
```

- `## main...origin/main [behind N]` — GitHub has commits you do not have; **pull** before you build on top of old code.
  - `## main...origin/main [ahead N]` — you have local commits not on GitHub; **push** when ready.
  - `M` / `??` — modified or untracked files; decide what goes in the next commit.
- 

## 2. Golden habit (saves 80% of pain)

---

When you open the repo on **any** machine (Mac, Rivanna, Terminus on your phone):

```
cd /path/to/Ivy_Net
git pull origin main
```

Then edit. When you finish a **logical chunk**:

```
git status
git add <files-you-mean-it-for> # see §9 – not always `git add .`
git commit -m "Short present-tense description of what changed"
git push origin main
```

---

## 3. Trick: “I forgot to pull” but I have uncommitted work

---

You edited files locally; **origin** moved; you do not want to lose anything.

**Safest pattern** (includes **untracked** files like new scripts — note `-u`):

```
git stash push -u -m "WIP before syncing origin/main"
git pull --rebase origin main
git stash pop
```

- If `stash pop` reports **conflicts**: fix the files (search for `<<<<<<`), then `git add` those files, then continue (commit when you are ready). **Do not** run `git stash drop` until you are sure the stash is no longer needed.
- List stashes: `git stash list`. Re-apply without dropping: `git stash apply stash@{0}`.

If you have no untracked files, you can omit `-u`, but `-u` is safer when you might have added new files.

---

## 4. Trick: “I forgot to pull” but I already committed locally

---

Your commits are only on this machine; GitHub has other commits too.

```
git pull --rebase origin main
# resolve conflicts if Git stops - fix files, git add ..., then:
git rebase --continue
git push origin main
```

`--rebase` **replays** your commits on top of the updated `main`, so history stays a straight line (usual team habit).

---

## 5. Trick: “What is on GitHub that I do not have?”

---

```
git fetch origin
git log --online HEAD..origin/main
```

Empty output means you are caught up (after a fetch). Non-empty lines are commits you would get with `pull`.

---

## 6. Trick: “The staging area looks wrong” (Dropbox / Cursor / accidental `git add`)

---

**Symptom:** `git status` shows weird **staged** deletions or moves you did not intend, often right after sync tools touched files.

**Unstage everything** (does **not** delete your file edits — only clears what is “ready to commit”):

```
git restore --staged .
```

Then `git status` again and `git add` only what you want in the next commit.

**Not the same as** `git reset --hard` (see §11).

---

## 7. Trick: “Throw away edits to one file” (working tree only)

---

You changed `foo.py` and want the **last committed** version back:

```
git restore path/to/foo.py
```

**Careful:** that **discards** uncommitted changes to that file.

---

## 8. Trick: Rename / move tracked files

---

Git usually detects renames if you stage both sides:

```
git mv old/path/file.md new/path/file.md
git commit -m "Move file.md under new/path"
```

If you moved with Finder, stage **delete** + **add**; Git may still show a rename in `git status`.

---

## 9. What to put in a commit (Ivy\_Net habits)

---

**Usually commit:** source code (`.py`, `.slurm`), small configs, Markdown docs you care about, notebooks you mean to share.

**Often gitignored here (do not fight it):** huge sweep outputs under `sports/outputs/.../rivanna_faithful_537/`, many `slurm-*.out` / `.err` patterns, local caches — see root `.gitignore`.

Think twice:

- **Editor / agent noise** — e.g. `.specstory/` churn. Either commit intentionally or `git restore` those paths.
- **Large binaries** (PDFs) — fine if they are real artifacts; avoid committing huge generated PDFs by accident.

Prefer `git add <paths>` over `git add .` when the tree is noisy.

---

## 10. Dropbox + Git (short warning)

---

Dropbox can touch files while Git is working. If `git status` suddenly looks impossible after a pull or commit:

1. Run `git restore --staged .` and re-read `git status`.
  2. If a command was interrupted, `git status` may mention **rebase in progress** — see Git’s hint (`git rebase --abort` to cancel, or fix and `--continue`).
-

## 11. Danger zone (know what you are doing)

| Command                                                         | Effect                                                                                                     |
|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>git reset --hard HEAD</code>                              | Throws away <b>all</b> uncommitted changes in tracked files.<br><b>Cannot undo</b> without backup.         |
| <code>git push --force</code> / <code>--force-with-lease</code> | Rewrites history on GitHub. Avoid on <b>shared</b> <code>main</code> unless you already use that workflow. |

## 12. Tiny glossary

| Term            | Plain English                                                             |
|-----------------|---------------------------------------------------------------------------|
| Working tree    | Your actual files on disk right now.                                      |
| Index / staging | What will go into the <b>next commit</b> after <code>git commit</code> .  |
| Commit          | Saved snapshot with a message.                                            |
| Branch          | A line of commits (here, usually <code>main</code> ).                     |
| origin          | Nickname for your GitHub remote.                                          |
| fetch           | Download history from GitHub <b>without</b> merging into your branch yet. |
| pull            | Fetch + merge (or rebase, if configured) into current branch.             |

## 13. HPC reminder (Rivanna)

- **Repo root** on the cluster should match GitHub before long runs: `git pull` in `~/Ivy_Net` (or your clone path).
- Root-level `sim_job.slurm` is **not** copied by `./scripts/rsync_push_to_hpc.sh sweep` alone — use `git` (or explicit `scp`) so the driver script stays in sync.

More operations detail: `../sports/documents/RIVANNA_RUNBOOK.md`.

## 14. Official references (when this cheat sheet is not enough)

- [Git documentation — Book](#)

- [GitHub: Resolving merge conflicts](#)
- 

Both Git guides live in [docs/](#). Elementary = mental model; this doc = **when my hands are on the keyboard**.